Software Test Plan

Bank Management System

Team E11

Meghna Sanjeev PES1UG24CS269

Mrinmayi Raman PES1UG24CS278

Advika Raj PES1UG24CS906

Manasi Vipin PES1UG24CS260

1. Introduction

1.1 Purpose

This Test Plan defines the verification approach for the Bank Management System requirements. It covers functional behaviour, security validation, persistence, usability and basic performance, while avoiding tests for external banking integrations because they are outside the project scope.

1.2 Test Objectives

- Verify that each implemented functional requirement behaves as specified.
- Verify employee and customer access restrictions.
- Verify invalid input does not cause an unintended state change.
- Verify account and transaction data persists across restart.
- Verify the simple console interface is usable and transaction response is within the specified local target.

1.3 Test Environment

Item	Planned Environment
Language	C/C++
Interface	Console
Compiler	GCC or equivalent C/C++ compiler
Storage	Local files
Execution	Single local machine; sample academic data

2. Test Items and Scope

2.1 Features to be Tested

- Employee login; create/search/update/close account.
- Customer authentication; deposit; withdrawal; balance; mini-statement.
- Unique account number generation.

- Input validation and error messages.
- File persistence.
- Console usability and local transaction response time.

2.2 Features Not to be Tested

- Real bank integration, NEFT/IMPS/UPI or payment gateways.
- Network security or remote access.
- Database performance.
- Multi-user concurrency.
- Production-grade encryption or regulatory compliance.

3. Test Strategy / Features and Approach

3.1 Functional Testing

Black-box functional tests will use valid, invalid and boundary inputs. Expected outputs are compared with the SRS requirement and the account/transaction state after each operation.

3.2 Security Validation

Security validation will verify that administrative operations cannot be reached without employee authentication, customer operations cannot be reached without customer authentication, failed credentials do not execute the requested operation, and non-positive transaction amounts are rejected.

3.3 Non-Functional Testing

Performance will be checked with a stopwatch or simple timing measurement for representative local transactions. Usability and maintainability will be checked through interface review and code review. Persistence will be verified by restarting the application.

3.4 Entry Criteria

- Program compiles successfully.
- Required sample data/files are available.
- Test build can start and display the main menu.

3.5 Exit Criteria

- All high-priority tests pass.
- No unresolved defect prevents core banking operations.
- Security access-control tests pass.
- Persistence test passes.

3.6 Pass/Fail Criteria

A test passes when the observed result and post-condition match the expected result. A test fails when the system performs an unintended operation, produces incorrect data, bypasses an access restriction, or exceeds a stated measurable requirement.

4. Test Cases

ID	Requirement	Test	Input/Precondition	Expected Result	Type
TC-01	FR-1, FR-2	Create account with valid details	Employee logged in; valid name/address/contact; initial deposit ₹1000	Account created; unique account number shown; balance = ₹1000	Functional
TC-02	FR-1, FR-12	Create account with invalid/duplicate data	Use duplicate account data or invalid required field	Account is not incorrectly created; clear error shown	Functional
TC-03	FR-3	Deposit valid amount	Authenticated customer; balance ₹1000; deposit ₹500	Balance becomes ₹1500; transaction recorded	Functional
TC-04	FR-3, FR-12, SEC-4	Deposit zero/negative amount	Authenticated customer; enter 0 or negative amount	Deposit rejected; balance unchanged; error shown	Security/Functional
TC-05	FR-4	Withdraw within balance	Authenticated customer; balance ₹1500; withdraw ₹500	Balance becomes ₹1000; transaction recorded	Functional
TC-06	FR-4, FR-12	Withdraw greater than balance	Authenticated customer; balance ₹1000; withdraw ₹1500	Withdrawal rejected; balance unchanged; error shown	Functional
TC-07	FR-5	Check balance	Authenticated customer	Displayed balance matches stored balance	Functional
TC-08	FR-6	View mini-statement	Authenticated customer with prior transactions	Recent transactions displayed in readable order	Functional
TC-09	FR-7	Search account	Employee logged in; enter valid account number	Correct account details displayed	Functional
TC-10	FR-8	Update account details	Employee logged in; modify contact/address	Updated details saved and displayed on re-check	Functional
TC-11	FR-9	Close account	Employee logged in; choose an existing account	Account marked/removed according to implementation; further customer operations are blocked	Functional
TC-12	FR-10, SEC-1, SEC-3	Employee authentication	Try correct and incorrect employee credentials	Correct credentials allow admin menu;	Security

				incorrect credentials deny access	
TC-13	FR-11, SEC-2, SEC-3	Customer authentication	Try correct and incorrect account no./PIN	Correct credentials allow self-service; incorrect credentials deny access	Security
TC-14	FR-12, NFR-3	Invalid input handling	Enter non-numeric menu/amount or invalid account number	Clear error; no unintended record/balance change	Reliability
TC-15	FR-13, NFR-6	Persistence across restart	Create/update account; close program; restart	Saved account/transaction information remains available	Persistence
TC-16	NFR-1	Transaction response time	Run representative deposit/withdrawal/balance operations locally	Each operation completes within 2 seconds under normal test conditions	Performance
TC-17	NFR-2	Console usability review	Run complete menu flow as a basic user	Menus, prompts and errors are understandable without special training	Usability
TC-18	NFR-4, NFR-5	Build/code review	Compile on target development OS; inspect module structure	Build succeeds; major features separated into modules/functions	Maintainability/Portability

5. Test Procedure and Reporting

5.1 Security Validation Procedure

1. Start the program and attempt to access employee administrative operations without logging in; access must be denied.
2. Enter invalid employee credentials; no administrative operation may execute.
3. Log in with valid employee credentials and verify that create/search/update/close operations are available.
4. Attempt customer operations with invalid account number/PIN; access must be denied.
5. Log in as a valid customer and verify deposit/withdrawal/balance/mini-statement are available.
6. Attempt zero or negative transaction amounts; the system must reject them without changing the balance.

5.2 Defect Reporting

Field	Description
Test Case ID	Identifier of failed test
Requirement ID	Affected SRS requirement
Steps	Short reproduction steps
Expected	Expected result from SRS

Actual	Observed result
Severity	High / Medium / Low
Status	Open / Fixed / Retest / Closed

5.3 Test Traceability

The Test Case column in the SRS RTM links every functional, non-functional and security requirement to one or more tests. This gives bidirectional traceability: requirements can be verified through tests, and test cases can be justified by requirements.

6. Risks and Mitigation

Risk	Mitigation
File corruption during manual testing	Use copied sample data and make backups before test runs.
Incorrect test state due to previous runs	Reset/recreate sample files before independent test cases.
Timing varies by machine	Measure several local runs and record the test environment.
Scope creep	Do not test or promise networking, external APIs, databases or real banking integrations.